Screen Recording Walkthrough Script

My Screen Recording Walkthrough Script

> ***How to use this document:***
> *Keep this document open on your second monitor or on your phone while recording your screen with Loom or OBS.*
> *Each step tells you **what to show on your screen** and gives you the **exact simple words to say**.*

Quick Setup Before Starting (1 minute check)

[ ] Face camera is enabled and visible in the corner.

[ ] Browser open to `http://localhost:8000` (or have your terminal ready).

[ ] VS Code open with these files ready in the tab bar:

- `config.py`
- `generator.py`
- `evaluator.py`
- `orchestrator.py`
- `memory.py`
- `logs/rejection_log.json`

[ ] Take a deep breath and speak in a calm, confident, conversational voice!

Step 1: Introduction & Philosophy (Time: 0:00 – 2:00)

■■ What to show:

Show the Web UI Dashboard at `http://localhost:8000` (or the VS Code project tree).

■■ What to say:

> *"Hello! My name is [Your Name], and today I am walking you through my submission for the Self-Evaluating Lesson Content Generator.*
> *The goal of this project is to build an autonomous agent that writes beginner-friendly lessons on technical topics—specifically on our submission topic: **'Introduction to RAG (Retrieval-Augmented Generation)'**.*
> *The core philosophy of this project is simple:*
> ***We never trust an LLM's own self-reported score.***
> *Often, if you ask an LLM 'Did you write a good lesson?', it will simply say 'Yes, I did great!' even if the lesson is full of dense jargon.*
> *In this project, we enforce an explicit, evidence-based quality contract:*
> *Every draft is evaluated by a strict grader against **six hard pass/fail checkpoints**. Python code deterministically verifies that **all six checkpoints pass** before anything is allowed to ship.*
> *If a draft fails, the agent captures structured feedback, writes a rejection log, and regenerates with targeted fixes within a capped retry limit.*
> *Let's see this running live!"*

Step 2: Live Demo — Clean Passing Run (Time: 2:00 – 5:30)

■■ What to show:

On `http://localhost:8000`:

1. Leave the checkbox **"Deliberate Error Demo" UNCHECKED**.
2. Point your mouse to the topic: *"Introduction to RAG (Retrieval-Augmented Generation)"*.
3. Click **"Generate Lesson"**.

■■ What to say:

> *"First, let's run a clean generation pipeline for our primary topic: Introduction to RAG.*
> *(Click 'Generate Lesson')*
> *On the left panel, the agent is executing live:*
> *First, it checks our cross-run memory for any past failure patterns.*
> *Then, it writes the lesson using our beginner pedagogy rules.*
> *And finally, it passes the text to our strict evaluator.*
> *(Point to the scorecard appearing on the left panel)*
> *Look at the six rubric checkpoints evaluated on Attempt 1:*
> *1. **Technical Accuracy & Grounding (PASS):** Verifies that RAG accurately describes retrieving documents to help an LLM answer questions without making things up.*
> *2. **Beginner-Friendly Language (PASS):** Verifies that sentences are short and simple for someone with limited technical vocabulary.*
> *3. **Teaches by Example (PASS):** Verifies that it includes concrete analogies—here it uses the 'open-book exam' analogy and the 'hotel swimming pool' customer scenario.*
> *4. **No Unexplained Jargon (PASS):** Verifies that terms like AI, retrieval, and hallucinations are defined in plain words before use.*
> *5. **Covers Key Points (PASS):** Verifies that What it is, Why it matters, and How it works are all present.*
> *6. **Coherent Teaching Flow (PASS):** Verifies a clear beginning, middle, and recap takeaway.*
> *Because all six are green, our Python code confirms **PASSED (Ship Ready)** on Attempt 1.*
> *(Point to the right panel)*
> *On the right panel, here is our finished lesson rendered in Markdown.*
> *We have two export options:*
> *I can click **'Copy Text'** to paste it directly into my Google Doc submission, or click **'Download PDF'** to instantly download a clean, white-background PDF for offline reading."*

Step 3: Deliberate Error Demo — The Evaluator Catch (Time: 5:30 – 9:30)

■■ What to show:

1. Check the box: **"Deliberate Error Demo (force initial jargon failure)"**.
2. Click **"Generate Lesson"**.

■■ What to say:

> *"Now, the assessment requires demonstrating the evaluator catching an intentional defect and the agent self-healing via feedback.*
> *To demonstrate this, I am checking the **'Deliberate Error Demo'** toggle. This intentionally forces Attempt 1 to draft the lesson using dense, unexplained academic jargon.*
> *(Click 'Generate Lesson')*
> *Watch Attempt 1 on the screen:*
> *The evaluator caught the defects immediately!*
> *Notice that **'Beginner-Friendly Language'** and **'No Unexplained Jargon'** have failed with red badges.*
> *The evaluator specifically critiqued the draft, noting that it used terms like 'parametric memory' and 'epistemic closure' without explaining them to a beginner.*
> *(Point to the amber feedback box below Attempt 1)*
> *Look right here below Attempt 1: the agent automatically synthesized a **Corrective Feedback Box**. It explicitly passed these failure reasons into the prompt for Attempt 2.*
> *Now look at Attempt 2:*
> *The generator read that feedback, rewrote the lesson in plain English with everyday analogies, and Attempt 2 passed all six checks cleanly!*
> *(Switch to VS Code and open `logs/rejection_log.json`)*
> *Let's look at `logs/rejection_log.json` in VS Code:*
> *Here is the real structured rejection record written to disk. It records: what failed, the exact reasons from the evaluator, and what changed on the next try. This gives us full auditability."*

Step 4: Code Architecture Walkthrough (Time: 9:30 – 14:00)

■■ What to show:

In VS Code, open each file in order and highlight the key lines.

1. `config.py`

> *"Let's look at how the code is structured. We'll start with `config.py`.*
> *Here we configure our model: Google's `gemini-3.5-flash-lite`, which is super fast (~1.5s response time) and reliable, along with automatic fallback models to ensure zero crashes from API rate limits.*
> *We also set `MAX_RETRIES = 2` so our retry loop always terminates."*

2. `generator.py`

> *(Open `generator.py` and scroll to `build_generator_prompt`)*
> *"In `generator.py`, we construct our prompt. Notice the strict pedagogy rules: zero assumed technical background, simple everyday words, and covering What, Why, and How.*
> *Notice lines 35–45: when a previous attempt fails, the feedback is prepended directly to the prompt under 'CRITICAL CORRECTIONS', forcing the model to fix every flagged issue on retry."*

3. `evaluator.py`

> *(Open `evaluator.py` and scroll to `validate_evaluation_dict`)*
> *"In `evaluator.py`, we call Gemini with a structured JSON prompt instructing it to grade each of the six rubric checkpoints with a binary `true` or `false` plus a reason.*
> *Notice lines 60–65: `overall_pass` is computed deterministically in Python using `all(...)`. We never trust the model's own self-reported verdict—only code decides if a lesson ships.*
> *We also extract JSON defensively to handle any code fences or stray markdown."*

4. `memory.py`

> *(Open `memory.py`)*
> *"In `memory.py`, we fulfill two requirements:*
> *First, per-run persistence: `save_run` writes full attempt histories to `data/runs/`, and `append_rejection_log` updates `logs/rejection_log.json`.*
> *Second, cross-run learning: `update_memory_from_runs` scans all past runs in `data/runs/`. If any checkpoint repeatedly fails across runs, it generates standing instructions in `data/memory.json`.*
> *`generator.py` loads these notes on future runs, so the agent learns from past mistakes over time."*

5. `orchestrator.py` & `main.py`

> *(Open `orchestrator.py`)*
> *"Finally, `orchestrator.py` runs the procedural while-loop: generate ➔ evaluate ➔ append feedback if failed ➔ retry (up to MAX_RETRIES) ➔ write rejection logs and save the final lesson to `data/runs/final_lesson.md` and `final_lesson.pdf`.*
> *And `main.py` provides the standard CLI entry point: `python main.py 'Introduction to RAG'`."*

Step 5: Conclusion & Wrap-up (Time: 14:00 – 15:00)

■■ What to show:

Show the final lesson on screen (or show `data/runs/final_lesson.md`).

■■ What to say:

> *"To wrap up, we have built and verified:*
> *1. A clean end-to-end run producing a beginner-friendly lesson on 'Introduction to RAG'.*
> *2. An evaluator that catches deliberate errors and guides self-healing retries.*
> *3. Structured rejection logs and cross-run memory that sharpen future prompts.*
> *4. Clean outputs in Markdown, JSON, and PDF ready for assessment submission.*
> *All deliverables are present in the repository, and the setup requires only a standard Gemini API key.*
> *Thank you very much for your time and for reviewing my project!"*

Helpful Tips for the Recording

- **Pacing:** Don't rush! Speaking at a calm, natural conversational pace makes it easy to follow and helps you hit the 12–15 minute mark effortlessly.

- **Mouse Clicks:** When you click "Generate Lesson", highlight what's happening with your mouse so the viewer can follow your actions.
- **If You Make a Mistake:** Don't restart the whole video! Just pause for 2 seconds, say *"Let me rephrase that..."*, and keep going. Reviewers appreciate authentic, clear communication.